

Numerical Methods

Python Code Reference

Every method, one script each — companion to the full lecture notes

BS–MS Physics, IISER Thiruvananthapuram

September 19, 2026

This is a standalone, code-only companion to the main lecture notes. Every numerical method covered there is given here as a single self-contained, runnable Python script (NumPy only), with a one-line description and its output on the worked example used in the notes. Methods with an accompanying MATLAB/Jupyter file in the original course material are marked accordingly. Copy any block directly into a `.py` file or a notebook cell — each is independent and needs only `numpy` (and `matplotlib` for the two plotting scripts).

Contents

1	Root-Finding Methods	3
1.1	Bisection Method	3
1.2	Newton–Raphson Method	3
1.3	Modified Newton–Raphson Method (Known Multiplicity)	4
1.4	Secant Method	4
1.5	Inverse Quadratic Interpolation (IQI)	5
2	Interpolation	6
2.1	Lagrange Interpolation	6
2.2	Newton’s Divided-Difference Interpolation	6
2.3	Cubic Spline Interpolation (Natural)	7
3	Numerical Integration	9
3.1	Trapezoidal Rule (Single Panel)	9
3.2	Composite Trapezoidal Rule	9
3.3	Simpson’s Rule (Single Panel Pair)	9
3.4	Composite Simpson’s Rule	10
3.5	Midpoint Rule (Single Panel)	10
3.6	Composite Midpoint Rule	11
3.7	Gaussian Quadrature	11
4	Linear Systems & Newton for Systems	12
4.1	LU Decomposition with Partial Pivoting	12
4.2	Newton’s Method for Systems of Nonlinear Equations	13

Chapter 1

Root-Finding Methods

1.1 Bisection Method

Finds a root of $f(x) = 0$ by repeatedly halving a bracketing interval $[a, b]$ with $f(a)f(b) < 0$. Linear convergence, guaranteed to converge.

```
1 import numpy as np
2
3 def f(x):
4     return x**3 + x - 1
5
6 a, b = 0.1, 1
7 tol = 1e-8
8 clist = []
9
10 if f(a) * f(b) > 0:
11     print('choose different points')
12 else:
13     while np.abs(b - a) / 2 > tol:
14         c = 0.5 * (a + b)
15         clist.append(c)
16         if f(a) * f(c) < 0:
17             b = c
18         else:
19             a = c
20     print('root =', c)
21     print('f(c) =', f(c))
```

Example output

root = 0.6823278... ($f(x) = x^3 + x - 1$ on $[0.1, 1]$)

1.2 Newton–Raphson Method

Uses the tangent line $x_{i+1} = x_i - f(x_i)/f'(x_i)$. Quadratic convergence near a simple root; needs f' and a decent initial guess.

```
1 import numpy as np
2
3 def f(x):
4     return x**3 + x - 1
5
6 def fd(x):          # derivative
7     return 3*x**2 + 1
8
9 x0 = 0.1
10 tol = 1e-8
```

```

11 val = 1
12 xs = []
13
14 while np.abs(val) > tol:
15     x = x0 - f(x0)/fd(x0)
16     val = x - x0
17     x0 = x
18     xs.append(x0)
19
20 print('root =', x)

```

Example output

root = 0.68232780... (same f , $x_0 = 0.1$, 7–8 iterations to converge)

1.3 Modified Newton–Raphson Method (Known Multiplicity)

When a root has multiplicity $m > 1$, plain Newton converges only linearly; scaling the step by m restores quadratic convergence: $x_{i+1} = x_i - m f(x_i)/f'(x_i)$.

```

1 import numpy as np
2
3 def f(x):
4     return 36*x**4 - 12*x**3 + 37*x**2 - 12*x + 1
5
6 def fd(x):
7     return 144*x**3 - 36*x**2 + 74*x - 12
8
9 def modified_newton(f, fd, x0, m, tol=1e-10, maxit=100):
10     """Modified Newton–Raphson for a root of known multiplicity m.
11     Restores quadratic convergence:  $x_{i+1} = x_i - m*f(x_i)/f'(x_i)$ ."""
12     x = x0
13     for _ in range(maxit):
14         fx = f(x)
15         xn = x - m * fx / fd(x)
16         if abs(xn - x) < tol:
17             return xn
18         x = xn
19     return x
20
21 m = 2 # known multiplicity of the root  $r = 1/6$ 
22 root = modified_newton(f, fd, 0.3, m)
23 print("root =", root)

```

Example output

root = 0.16666667 ($f(x) = 36x^4 - 12x^3 + 37x^2 - 12x + 1 = (6x - 1)^2(x^2 + 1)$, multiplicity $m = 2$ root at $x = 1/6$; reaches machine precision in 3–4 iterations versus $\sim 12+$ for plain Newton)

1.4 Secant Method

Replaces $f'(x_i)$ with a finite-difference slope through the last two iterates — no derivative needed. Superlinear convergence, order ≈ 1.618 .

```

1 import numpy as np
2
3 def f(x):
4     return x**3 + x - 1
5
6 x0, x1 = 0.1, 0.2
7 tol = 1e-8

```

```

8  val = 1
9  xs = []
10
11 while np.abs(val) > tol:
12     fd = (f(x1) - f(x0)) / (x1 - x0)    # secant slope
13     x = x1 - f(x1)/fd
14     val = x - x1
15     xs.append(x)
16     x0, x1 = x1, x
17
18 print('root =', x1)

```

Example output

root = 0.68232780... (same f , $x_0 = 0.1$, $x_1 = 0.2$)

1.5 Inverse Quadratic Interpolation (IQI)

Fits a quadratic through the last three iterates (as $x = P(y)$) and evaluates it at $y = 0$. Superlinear convergence, order ≈ 1.84 ; derivative-free.

```

1  import numpy as np
2
3  def f(x):
4      return x**3 + x - 1
5
6  x0, x1, x2 = 0.1, 0.15, 0.25
7  tol = 1e-8
8  val = 1
9  xs = []
10
11 while np.abs(val) > tol:
12     q = f(x0)/f(x1)
13     r = f(x2)/f(x1)
14     s = f(x2)/f(x0)
15     x3 = x2 - (r*(r-q)*(x2-x1) + s*(1-r)*(x2-x0)) / ((q-1)*(r-1)*(s-1))
16     xs.append(x3)
17     val = x3 - x2
18     x0, x1, x2 = x1, x2, x3
19
20 print('root =', x2)

```

Example output

root = 0.68232780... (same f , $x_0 = 0.1$, $x_1 = 0.15$, $x_2 = 0.25$)

Chapter 2

Interpolation

2.1 Lagrange Interpolation

Builds the unique degree- $(n - 1)$ interpolating polynomial directly as a weighted sum of Lagrange basis polynomials $l_i(x)$, without solving any linear system.

```
1 import numpy as np
2
3 def lagrange_interp(xdata, ydata, x):
4     """Evaluate the Lagrange interpolating polynomial at point x."""
5     n = len(xdata)
6     total = 0.0
7     for i in range(n):
8         term = ydata[i]
9         for j in range(n):
10             if j != i:
11                 term *= (x - xdata[j]) / (xdata[i] - xdata[j])
12         total += term
13     return total
14
15 xdata = [0, 2, 3]
16 ydata = [1, 3, 0]
17
18 print("P(1) =", lagrange_interp(xdata, ydata, 1))
```

Example output

P(1) = 3.33333333 (through (0, 1), (2, 3), (3, 0) — matches Chapter 2's example / Problem Set Q3(a))

2.2 Newton's Divided-Difference Interpolation

Builds the same interpolating polynomial recursively via the divided-difference table; more efficient to *extend* with new nodes than Lagrange, since old coefficients don't need to be recomputed.

```
1 import numpy as np
2
3 def divided_diff(xdata, ydata):
4     """Return the Newton divided-difference coefficients f[x0], f[x0 x1], ..."""
5     n = len(xdata)
6     coef = np.array(ydata, dtype=float).copy()
7     for j in range(1, n):
8         coef[j:n] = (coef[j:n] - coef[j-1:n-1]) / \
9                     (np.array(xdata[j:n]) - np.array(xdata[0:n-j]))
10    return coef
11
12 def newton_eval(xdata, coef, x):
13     """Evaluate the Newton-form interpolating polynomial at point x
```

```

14     (Horner-like nested evaluation)."""
15     n = len(coef)
16     p = coef[n - 1]
17     for k in range(1, n):
18         p = coef[n - 1 - k] + (x - xdata[n - 1 - k]) * p
19     return p
20
21 xdata = [0, 2, 3]
22 ydata = [1, 3, 0]
23
24 coef = divided_diff(xdata, ydata)
25 print("divided differences =", coef)
26 print("P(1) =", newton_eval(xdata, coef, 1))

```

Example output

```

divided differences = [1.0, 1.0, -1.333...]
P(1) = 3.33333333 (identical to the Lagrange result above, as guaranteed by the Uniqueness Theo-
rem)

```

2.3 Cubic Spline Interpolation (Natural)

Builds a piecewise-cubic interpolant with continuous value, slope, and curvature at every interior node, by solving a tridiagonal system for the curvature coefficients c_i (natural boundary conditions $S_1''(x_1) = S_{n-1}''(x_n) = 0$).

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def spline_coeff(x, y):
5      n = len(x)
6      dx = np.diff(x)
7      dy = np.diff(y)
8
9      A = np.zeros((n, n))
10     r = np.zeros(n)
11     for i in range(1, n - 1):
12         A[i, i-1:i+2] = [dx[i-1], 2*(dx[i-1]+dx[i]), dx[i]]
13         r[i] = 3*(dy[i]/dx[i] - dy[i-1]/dx[i-1])
14
15     # Natural spline endpoint conditions: S''(x_1) = S''(x_n) = 0
16     A[0, 0] = 1
17     A[-1, -1] = 1
18
19     c = np.linalg.solve(A, r)
20
21     coeff = np.zeros((n, 3))      # columns: [b, c, d]
22     for i in range(n - 1):
23         coeff[i, 2] = (c[i+1] - c[i]) / (3*dx[i])
24         coeff[i, 0] = dy[i]/dx[i] - dx[i]*(2*c[i] + c[i+1]) / 3
25         coeff[i, 1] = c[i]
26     return coeff
27
28 x = np.array([0, 1, 2])
29 y = np.array([3, -2, 1])
30 coeff = spline_coeff(x, y)
31
32 N = 10
33 xs1 = np.linspace(x[0], x[1], N)
34 xs2 = np.linspace(x[1], x[2], N)
35 ys1 = y[0] + coeff[0,0]*(xs1-x[0]) + coeff[0,1]*(xs1-x[0])**2 + coeff[0,2]*(xs1-x[0])
36     **3
37 ys2 = y[1] + coeff[1,0]*(xs2-x[1]) + coeff[1,1]*(xs2-x[1])**2 + coeff[1,2]*(xs2-x[1])
38     **3

```

```
38 plt.plot(x, y, "o", label="Data points")
39 plt.plot(np.concatenate((xs1,xs2)), np.concatenate((ys1,ys2)), "-", label="Cubic
    spline")
40 plt.legend(); plt.grid(True); plt.show()
```

Example output

Plots the spline through $(0, 3), (1, -2), (2, 1)$; see Chapter 2 and Problem Set Q8 for worked coefficient tables.

Chapter 3

Numerical Integration

3.1 Trapezoidal Rule (Single Panel)

Approximates $\int_{x_0}^{x_1} f(x) dx$ by the area of the trapezoid under the straight line through the two endpoints. Local error $O(h^3)$; exact for linear f .

```
1 def trapezoidal(f, x0, x1):
2     """Single-panel trapezoidal rule."""
3     h = x1 - x0
4     return h / 2 * (f(x0) + f(x1))
5
6 f = lambda x: x**2
7 print(trapezoidal(f, 0, 1))
```

Example output

```
trapezoidal(lambda x: x**2, 0, 1) = 0.5    (exact value 1/3; error = 1/6, matching Chapter 3's
worked example)
```

3.2 Composite Trapezoidal Rule

Approximates f by straight lines on each of m panels. Global error $O(h^2)$.

```
1 import numpy as np
2
3 def composite_trapezoidal(f, a, b, m):
4     """Composite trapezoidal rule with m panels."""
5     h = (b - a) / m
6     x = np.linspace(a, b, m + 1)
7     return h * (f(x[0]) / 2 + f(x[-1]) / 2 + np.sum(f(x[1:-1])))
8
9 f = lambda x: x**2
10 print(composite_trapezoidal(f, 0, 1, 1))    # 1 panel: h=1
11 print(composite_trapezoidal(f, 0, 1, 100)) # 100 panels: converges to 1/3
```

Example output

```
composite_trapezoidal(lambda x: x**2, 0, 1, 1) = 0.5    (exact value 1/3; matches the hand-
worked example in Chapter 3)
```

3.3 Simpson's Rule (Single Panel Pair)

Approximates $\int_{x_0}^{x_2} f(x) dx$ by the area under the parabola through the two endpoints and the midpoint. Local error $O(h^5)$; exact for cubics (degree of precision 3).

```

1 def simpson(f, x0, x2):
2     """Single-application Simpson's rule (one parabola over [x0, x2])."""
3     h = (x2 - x0) / 2
4     x1 = (x0 + x2) / 2      # midpoint
5     return h / 3 * (f(x0) + 4 * f(x1) + f(x2))
6
7 f = lambda x: x**2
8 print(simpson(f, 0, 1))

```

Example output

`simpson(lambda x: x**2, 0, 1) = 0.33333333` (exact — matches Chapter 3's worked example: error is zero since f is a quadratic)

3.4 Composite Simpson's Rule

Approximates f by parabolas on pairs of panels (needs an even number of panels). Global error $O(h^4)$ — exact for cubics.

```

1 import numpy as np
2
3 def composite_simpson(f, a, b, m):
4     """Composite Simpson's rule; m must be even (m panels -> m/2 parabolas)."""
5     if m % 2 != 0:
6         raise ValueError("m (number of panels) must be even")
7     h = (b - a) / m
8     x = np.linspace(a, b, m + 1)
9     y = f(x)
10    return h / 3 * (y[0] + y[-1]
11                  + 4 * np.sum(y[1:-1:2])    # odd-indexed nodes: weight 4
12                  + 2 * np.sum(y[2:-1:2]))    # even-indexed interior: weight 2
13
14 f = lambda x: x**2
15 print(composite_simpson(f, 0, 1, 2))    # exact for this quadratic: 1/3

```

Example output

`composite_simpson(lambda x: x**2, 0, 1, 2) = 0.33333333` (exact — matches Chapter 3's degree-of-precision-3 result)

3.5 Midpoint Rule (Single Panel)

Approximates $\int_{x_0}^{x_1} f(x) dx$ by the rectangle of height $f(m)$ at the panel's midpoint m . Local error $O(h^3)$, with error constant half that of the trapezoidal rule and opposite in sign.

```

1 def midpoint(f, x0, x1):
2     """Single-panel midpoint rule."""
3     h = x1 - x0
4     m = (x0 + x1) / 2
5     return h * f(m)
6
7 f = lambda x: x**2
8 print(midpoint(f, 0, 1))

```

Example output

`midpoint(lambda x: x**2, 0, 1) = 0.25` (exact value $1/3$; error = $-1/12$, half the trapezoidal error and opposite sign, as derived in Chapter 3)

3.6 Composite Midpoint Rule

Approximates f by its value at each panel's midpoint. Never evaluates f at the endpoints — useful for integrands singular there.

```

1 import numpy as np
2
3 def composite_midpoint(f, a, b, n):
4     """Composite midpoint rule with n panels; never evaluates f at a or b."""
5     h = (b - a) / n
6     midpoints = a + h/2 + h * np.arange(n)
7     return h * np.sum(f(midpoints))
8
9 f = lambda x: x**2
10 print(composite_midpoint(f, 0, 1, 1))
11 print(composite_midpoint(f, 0, 1, 100)) # converges to 1/3
12
13 # Example use case: integrand singular at an endpoint
14 g = lambda x: np.sin(x) / x
15 print(composite_midpoint(g, 0, 1, 1000)) # sidesteps the x=0 singularity

```

Example output

```
composite_midpoint(lambda x: x**2, 0, 1, 1) = 0.25
```

3.7 Gaussian Quadrature

Uses the optimal (Gauss–Legendre) nodes and weights, computed via `numpy.polynomial.legendre.leggauss`, so an n -point rule is exact up to degree $2n - 1$.

```

1 import numpy as np
2
3 def gauss_quad(f, a, b, n):
4     """n-point Gauss-Legendre quadrature on [a, b].
5     Exact for polynomials up to degree 2n-1."""
6     t, c = np.polynomial.legendre.leggauss(n) # nodes/weights on [-1, 1]
7     x = 0.5 * (b - a) * t + 0.5 * (b + a)     # map to [a, b]
8     return 0.5 * (b - a) * np.sum(c * f(x))
9
10 f = lambda x: np.exp(-x**2 / 2)
11 print("n=2:", gauss_quad(f, -1, 1, 2))
12 print("n=4:", gauss_quad(f, -1, 1, 4))
13 print("n=8:", gauss_quad(f, -1, 1, 8)) # true value: 1.71124878378430

```

Example output

```

gauss_quad(lambda x: np.exp(-x**2/2), -1, 1, 2) = 1.69296345
gauss_quad(lambda x: np.exp(-x**2/2), -1, 1, 4) = 1.71122450
(true value 1.71124878 — matches the worked example in Chapter 3 exactly)

```

Chapter 4

Linear Systems and Newton's Method for Systems

4.1 LU Decomposition with Partial Pivoting

Factors $PA = LU$ using Gauss elimination, storing the elimination multipliers in L ; then solves $A\mathbf{x} = \mathbf{b}$ via two triangular solves. (Companion files: LU.m, LU.ipynb.)

```
1 import numpy as np
2
3 # Matrix A
4 a = np.array([[2, 1, 5], [4, 4, -4], [1, 3, 1]], dtype=float)
5 b = np.array([3, 3, -6])
6
7 L = np.eye(3)
8 n = a.shape[0]
9
10 for j in range(n - 1):
11     # Partial pivoting: find pivot row with largest |entry| in column j
12     I = j + np.argmax(np.abs(a[j:n, j]))
13     if I != j:
14         a[[j, I], :] = a[[I, j], :]
15
16     # Gaussian elimination
17     for i in range(j + 1, n):
18         mult = a[i, j] / a[j, j]
19         a[i, j] = mult # store multiplier -> becomes L[i, j]
20         for k in range(j + 1, n):
21             a[i, k] = a[i, k] - mult * a[j, k]
22
23 # Extract L and U from the worked-over array
24 U = np.zeros((3, 3))
25 for i in range(n):
26     for j in range(n):
27         if i > j:
28             L[i, j] = a[i, j]
29         else:
30             U[i, j] = a[i, j]
31
32 c = np.linalg.inv(L).dot(b) # step (a): forward substitution Lc = b
33 x = np.linalg.inv(U).dot(c) # step (b): back substitution Ux = c
34
35 print("L =\n", L)
36 print("U =\n", U)
37 print("x =\n", x)
```

Example output

```
L = [[1, 0, 0], [0.25, 1, 0], [0.5, -0.5, 1]]
U = [[4, 4, -4], [0, 2, 2], [0, 0, 8]]
x = [-1.96875, 1.921875, -0.796875]
```

4.2 Newton's Method for Systems of Nonlinear Equations

Solves $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ by linearizing with the Jacobian $J(\mathbf{x}_i)$ and solving $J(\mathbf{x}_i)\mathbf{s}_i = -\mathbf{F}(\mathbf{x}_i)$ (via Gauss elimination with partial pivoting) at every iteration. (Companion files: `Newton_SOE.m`, `Newton_SOE.ipynb`.)

```
1 import numpy as np
2
3 def func(x):
4     # returns -F(x), so that solving J s = -F(x) becomes J s = f below
5     return -1 * np.array([
6         x[1] - x[0]**3,
7         x[0]**2 + x[1]**2 - 1
8     ])
9
10 # Initial guess
11 x = np.array([1.0, 2.0])
12
13 # Jacobian matrix
14 a = np.array([[-3 * x[0]**2, 1], [2 * x[0], 2 * x[1]]], dtype=float)
15
16 # Function vector (already negated: this is -F(x))
17 f = func(x)
18
19 n = a.shape[0]          # number of variables
20 tol = n * 1e-2          # convergence tolerance
21 dx = np.ones(n)         # ||x_{i+1} - x_i||, initialized nonzero to enter the loop
22
23 while np.sum(dx) > tol:
24
25     xprev = x.copy()
26
27     # --- Gauss elimination with partial pivoting on [a | f] ---
28     for j in range(n - 1):
29         I = j + np.argmax(np.abs(a[j:n, j]))          # pick largest pivot
30         if I != j:
31             a[[j, I], :] = a[[I, j], :]              # swap rows of Jacobian
32             f[[j, I]] = f[[I, j]]                    # swap same rows of RHS
33         for i in range(j + 1, n):
34             mult = a[i, j] / a[j, j]
35             for k in range(j, n):
36                 a[i, k] = a[i, k] - mult * a[j, k]
37             f[i] = f[i] - mult * f[j]
38
39     # --- Back substitution: solve for the Newton step s ---
40     s = np.zeros(n)
41     s[n - 1] = f[n - 1] / a[n - 1, n - 1]
42     x[n - 1] = x[n - 1] + s[n - 1]
43
44     for i in range(n - 2, -1, -1):
45         SUM = 0.0
46         for j in range(i + 1, n):
47             SUM = SUM + a[i, j] * s[j]
48         s[i] = (f[i] - SUM) / a[i, i]
49         x[i] = x[i] + s[i]
50
51     # --- Re-evaluate Jacobian and function vector at the new iterate ---
52     a = np.array([
53         [-3 * x[0]**2, 1],
```

```
54         [2 * x[0], 2 * x[1]]
55     ], dtype=float)
56     f = func(x)
57
58     dx = np.abs(xprev - x)
59
60     print("Solution:")
61     print(x)          # [0.82604011  0.56361977]
```

Example output

Solution: [0.82604011 0.56361977] (intersection of $y = x^3$ and $x^2 + y^2 = 1$, from $\mathbf{x}_0 = (1, 2)$ in 4 iterations)